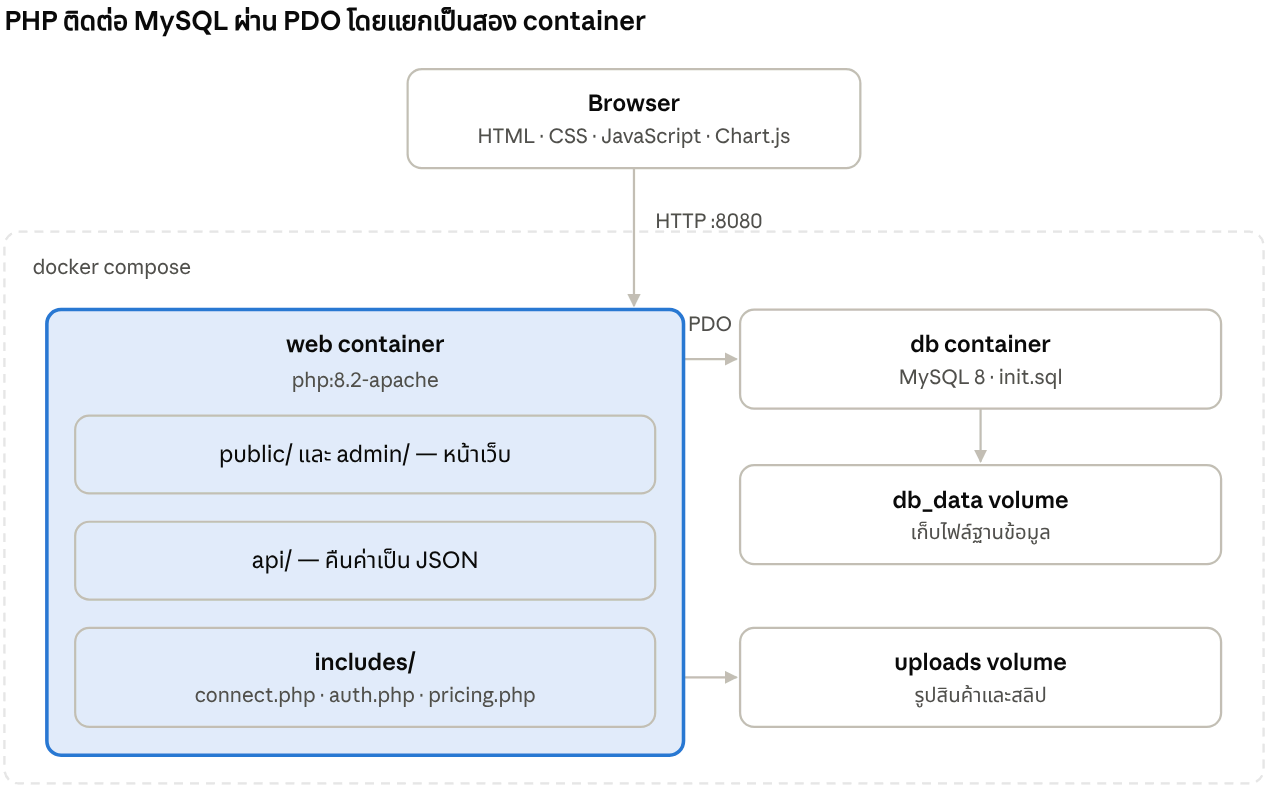
System Architecture — ร้านขายเสื้อออนไลน์

Oct 18, 2026 · @Sunday

ระบบเป็นเว็บแอปพลิเคชันแบบ 3 ชั้น เขียนด้วย PHP เชื่อมต่อ MySQL ผ่าน PDO รันบน Docker 2 container (web และ db) โดยแยก volume สำหรับฐานข้อมูลและไฟล์อัปโหลด

ภาพรวมสถาปัตยกรรม



ภาพรวมสถาปัตยกรรม · 2 container, 2 volume

เบราว์เซอร์ติดต่อกับ `web` container ผ่าน HTTP เท่านั้น ไม่ได้ต่อกับฐานข้อมูลโดยตรง โค้ด PHP จึงเป็นด่านเดียวที่เข้าถึงข้อมูลได้ และเป็นจุดที่ตรวจสิทธิ์ทุกครั้ง

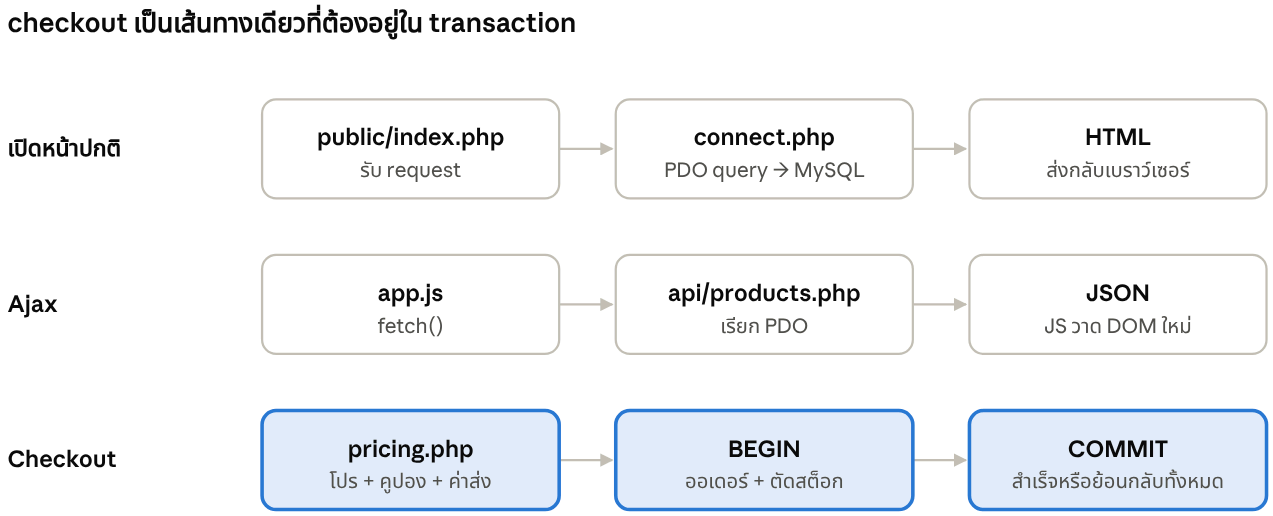
ชั้นของระบบ

ระบบแบ่งเป็น 3 ชั้น แต่ละชั้นรับผิดชอบคนละเรื่อง ทำให้แก้ส่วนหนึ่งโดยไม่กระทบอีกส่วนได้

ชั้น	หน้าที่	เทคโนโลยีที่ใช้
Client	แสดงผล ตรวจสอบฟอร์มเบื้องต้น เปลี่ยน DOM วาดกราฟ และยิง Ajax	HTML, CSS, JavaScript, Chart.js
Application	รับ request ตรวจ session และสิทธิ์ คิดราคา ติดต่อฐานข้อมูล แล้วส่ง HTML หรือ JSON กลับ	PHP 8.2 บน Apache, PDO
Data	เก็บข้อมูลทั้งหมดและไฟล์ที่ผู้ใช้อัปโหลด	MySQL 8, Docker volume

การตรวจสอบข้อมูลฝั่ง Client เป็นเพียงด่านแรกเพื่อความสะดวกของผู้ใช้ ชั้น Application ตรวจซ้ำทุกครั้งเสมอ เพราะค่าที่ส่งมาจากเบราว์เซอร์แก้ไขได้

เส้นทางของ request



เส้นทางของ request · 3 รูปแบบ

เส้นทางที่สองคือ Ajax เป็นส่วนที่ใช้ในหน้าค้นหาสินค้า การเพิ่มลงตะกร้า และการโหลดรายงานใหม่ โดยไม่ต้องรีโหลดทั้งหน้า

เส้นทางที่สามคือ checkout ต้องระวังเป็นพิเศษ เพราะการสร้างออเดอร์ การตัดสต็อก และการใช้คูปองต้องสำเร็จหรือล้มเหลวพร้อมกัน หากตัดสต็อกสำเร็จแต่สร้างออเดอร์ไม่สำเร็จ สินค้าจะหายจากคลังโดยไม่มีออเดอร์

โครงสร้างโฟลเดอร์

โฟลเดอร์แยกตามหน้าที่ ไม่ใช่ตามชื่อฟีเจอร์ ทำให้รู้ได้ทันทีว่าไฟล์ใหม่ควรไปอยู่ที่ไหน

โฟลเดอร์	เก็บอะไร	เหตุผลที่แยก
`public/`	หน้าสำหรับลูกค้า เช่น รายการสินค้า ตะกร้า ชำระเงิน ประวัติสั่งซื้อ	เป็นหน้าที่เปิดสาธารณะ ไม่ต้องตรวจสิทธิ์แอดมิน
`admin/`	หน้าสำหรับแอดมิน เช่น จัดการสินค้า ออเดอร์ โปรโมชั่น รายงาน	ทุกไฟล์ในนี้เรียก `auth.php` ตรวจสิทธิ์ก่อนเสมอ
`api/`	ไฟล์ที่คืนค่าเป็น JSON ให้ Ajax เรียกใช้	แยกจากหน้าปกติ ทำให้อธิบายและทดสอบได้ง่าย
`includes/`	`connect.php`, `auth.php`, `pricing.php`, `header.php`, `footer.php`	โค้ดที่ทุกหน้าใช้ร่วมกัน แก้ที่เดียวมีผลทั้งเว็บ
`assets/`	`style.css`, `app.js` และไลบรารีที่ดาวน์โหลดมาเก็บไว้	ไม่พึ่ง CDN ตอนนำเสนอ หากอินเทอร์เน็ตมีปัญหา
`uploads/`	รูปสินค้าและสลิปการชำระเงิน	ผูกกับ volume เพื่อไม่ให้ไฟล์หายเมื่อลบ container

ไฟล์สำคัญที่สุดคือ `includes/pricing.php` เพราะเป็นที่เดียวที่คำนวณราคา ทั้งโปรโมชั่น คูปอง และค่าจัดส่ง หน้าตะกร้า หน้าชำระเงิน และ API เรียกใช้ตัวเดียวกัน ราคาจึงไม่มีทางไม่ตรงกันระหว่างหน้า

อีกไฟล์ที่ช่วยประหยัดเวลาคือ `includes/header.php` เพราะวาง semantic layout และ responsive ไว้ที่เดียว ทุกหน้าที่ `include` จะได้โครงเดียวกันโดยไม่ต้องแก้ทีละหน้า

การ deploy ด้วย Docker

ทั้งระบบรันด้วยคำสั่งเดียวคือ `docker compose up` เครื่องที่ยังไม่เคยติดตั้งอะไรก็เปิดใช้งานได้ทันที

Service	Image	Port	หน้าที่
`web`	`php:8.2-apache`	8080	รันโค้ด PHP ทั้งหมดและเสิร์ฟหน้าเว็บ
`db`	`mysql:8`	ไม่เปิดออกภายนอก	เก็บฐานข้อมูล โหลด `init.sql` อัตโนมัติเมื่อสร้างครั้งแรก
`phpmyadmin`	`phpmyadmin`	8081	ใช้ตรวจข้อมูลระหว่างพัฒนา ปิดได้เมื่อส่งงาน

ทำไมต้องแยก container

หากรวม PHP กับ MySQL ไว้ container เดียว ทุกครั้งที่ build image ใหม่เพื่อแก้โค้ด ข้อมูลในฐานข้อมูลจะหายไปด้วย เมื่อแยกแล้วจะแก้โค้ดกี่รอบก็ไม่กระทบข้อมูล

ทำไมต้องมี volume

ข้อมูลที่เขียนไว้ใน container จะหายไปพร้อมกับ container เมื่อถูกลบ จึงต้องผูก volume ไว้สองก้อน

- `db_data` เก็บไฟล์ฐานข้อมูลของ MySQL
- `uploads` เก็บรูปสินค้าและสลิปที่ลูกค้าอัปโหลด

ข้อนี้เป็นจุดที่พลาดกันบ่อย เพราะระบบจะทำงานได้ปกติจนกว่าจะมีการลบหรือสร้าง container ใหม่ แล้วข้อมูลที่ใช้สาธิตจึงหายไปทั้งหมด

การตั้งค่าเชื่อมต่อ

`includes/connect.php` อ่านค่าเชื่อมต่อจาก environment variable ได้แก่ `DB_HOST`, `DB_NAME`, `DB_USER` และ `DB_PASS` โดยมีค่าตั้งต้นเป็นของ XAMPP ไฟล์เดียวกันจึงใช้ได้ทั้งบน XAMPP และบน Docker โดยไม่ต้องแก้โค้ด

ข้อจำกัดและสิ่งที่ยังไม่ได้รวม

สถาปัตยกรรมนี้ออกแบบให้พอดีกับขอบเขตของโปรเจกต์ จึงตัดบางส่วนที่ระบบจริงต้องมีออกไป

- **Session เก็บเป็นไฟล์ใน `web` container** หากลบ container ผู้ใช้ที่ล็อกอินอยู่จะหลุดทั้งหมด ซึ่งรับได้สำหรับโปรเจกต์นี้ ระบบจริงจะเก็บไว้ในฐานข้อมูลหรือ Redis
- **ยังไม่มี reverse proxy และ HTTPS** เพราะยังไม่ได้ตัดสินใจว่าจะนำขึ้นโฮสต์ที่ใด
- **การชำระเงินเป็นการจำลอง** ระบบสร้าง QR และรับสลิป แล้วให้แอดมินกดยืนยันเอง ไม่ได้เชื่อมต่อ API ธนาคารจริง
- **ไม่มีระบบส่งอีเมล** ฟังก์ชันลืมรหัสผ่านและการแจ้งเตือนจึงไม่อยู่ในขอบเขต
- **ไม่มีการทำสำเนาข้อมูลสำรองอัตโนมัติ** ข้อมูลอาศัย volume อย่างเดียว

รายการข้างต้นควรระบุไว้ในรายงานด้วย เพื่อแสดงว่าเป็นการตัดสินใจที่ตั้งใจ ไม่ใช่สิ่งที่มองข้าม